



Network Programming (IE2102)

2nd Year, 2nd Semester

Assignment

Design and Implementation of a Secure Multiprocessor TCP Application Server with Custom Protocol and Session Management

Submitted to

Sri Lanka Institute of Information Technology

IT24100368

Thaveesha L.H.K.

In partial fulfillment of the requirements for the
Bachelor of Science Special Honors Degree in Information Technology

04/10/2026

Important:

Remember

- *Plagiarism is considered as a serious crime against intellectual property in academic environments.*
- *By including the declaration page mentioned below, you abide by the laws of the institute to accept 0 marks for the report, in case you get caught attempting plagiarism.*

Annexure 1

General Formatting

1. Paper size: A4
2. Contents font & size: Times New Roman, size 12
3. Chapter headings/sub headings font & size: Times New Roman Bold, size 12
4. Page numbering:
 - Cover page should not contain any number
 - Declaration page to table of figures page should be numbered using Roman numbers at bottom center of the page
 - Content pages, references page(s) and annexes (if any) should be numbered using Arabic numbers at the bottom center of the page
5. Margins: 1 inch margins on left, right, top and bottom
6. Line spacing: 1 line spacing

Figure Numbering:

1. Each figure should be numbered at bottom center of the figure along with a figure name.
2. Figure number should start with word 'Figure' followed by the 'main chapter number' and 'figure number within the chapter' separated by a 'period sign (.)' and followed by a 'colon (:)'.

For example,

Figure 2.6: XYZ Connector

Declaration Page:

Declaration page should contain the following. Reports that does not contain this declaration 'AS IS' will not be accepted as valid reports and will be awarded 0 marks.

Declaration

I certify that this report does not incorporate without acknowledgement, any material previously submitted for a degree or diploma in any university, and to the best of my knowledge and belief it does not contain any material previously published or written by another person, except where due reference is made in text

Table of Contents

Table of figures	ii
Introduction	iii
Tasks and Screenshots	1
1. Hostname configuration	1
2. Network configuration	2
3. Server running (port proof using ss/netstat)	3
4. Successful REGISTER and LOGIN	3
5. Failed login + lockout proof	4
6. Multiple forked processes (ps output)	5
7. Log file entries	6
8. Proper child cleanup (no zombie processes)	7

Table of figures

<i>Figure 1.1: Host Configuration Details</i>	<i>1</i>
<i>Figure 2.1: Network configuration</i>	<i>2</i>
<i>Figure 3.1: Portt proof</i>	<i>2</i>
<i>Figure 4.1: Successfully register</i>	<i>3</i>
<i>Figure 4.2: Successfully login</i>	<i>3</i>
<i>Figure 5.1: Failed login + lockout proof</i>	<i>4</i>
<i>Figure 6.1: forked processes.</i>	<i>5</i>
<i>Figure 7.1: Comprehensive Server Audit Log (server_IT24102161.log).</i>	<i>6</i>
<i>Figure 8.1: Zombie Process check</i>	<i>7</i>

Introduction

This report is about the design and implementation of a Secure Multiprocessor TCP Application Server. I worked on this project for the IE2102 Network Programming module at the Sri Lanka Institute of Information Technology. My student number is IT24100368.

The main goal of this project was to create a client and server application that talks to each other using TCP. I wrote the server code in C. Used a thing called multiprocessing with the fork system call. The client code was written in Python. I also created a custom protocol for the application with message framing.

I set up the server to listen on port 50368. Every time the server responds it includes a Server ID, which is SID:1003. I got these numbers from my student registration number IT24100368. I named all the source files as required including server_0368.c, client_0368.py and Makefile_0368.

The project has five parts. The first part, Task A1 is about creating a custom TCP protocol with something called LEN framing. This means handling receives, oversized payloads and invalid lengths properly. The second part, Task A2 is about using multiprocessing with fork. The server can handle many clients at the same time without blocking.

Task A3 is about user authentication. I used commands like REGISTER, LOGIN and LOGOUT with a way of storing passwords called salted SHA-256 password hashing and session tokens. These tokens expire after 5 minutes of not using them. Task A4 is about protecting the Secure Multiprocessor TCP Application Server from abuse. This includes limiting how times someone can try to log in locking out people who try to log in many times checking if usernames are valid and rejecting payloads that are too big.

The last part, Task A5 is about keeping a record of everything that happens on the Secure Multiprocessor TCP Application Server. I made a log file called server_IT24100368.log that records every client action, including the time IP address, port, child process PID, username, command and result. This report shows all the features I implemented with screenshots from a demonstration, on an Ubuntu Linux machine running on a VMware Virtual Platform.

Tasks and Screenshots

1. Hostname configuration

```
kulani@kulani-VirtualBox:~/IE2102_IT24100368$  
kulani@kulani-VirtualBox:~/IE2102_IT24100368$ hostname  
kulani-VirtualBox  
  
kulani@kulani-VirtualBox:~/IE2102_IT24100368$  
kulani@kulani-VirtualBox:~/IE2102_IT24100368$ hostnamectl  
Static hostname: kulani-VirtualBox  
Icon name: computer-vm  
Chassis: vm   
Machine ID: 5879adf8c1474c6581ee6169e4535be8  
Boot ID: 073ffd36b24948099a4d1cb8c8103515  
Virtualization: oracle  
Operating System: Ubuntu 24.04.3 LTS  
Kernel: Linux 6.17.0-14-generic  
Architecture: x86-64  
Hardware Vendor: innotek GmbH  
Hardware Model: VirtualBox  
Firmware Version: VirtualBox  
Firmware Date: Fri 2006-12-01  
Firmware Age: 19y 4month 1w 3d  
kulani@kulani-VirtualBox:~/IE2102_IT24100368$
```

Figure 1.1: Host Configuration Details

Prior to setting up the server, the system environment was checked to confirm that the system and hostname were properly configured. This is a critical step in identifying a computer on a network and compatibility with a network programming environment.

2. Network configuration

```
kulani@kulani-VirtualBox:~/IE2102_IT24100368$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:ca:51:6c brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
        valid_lft 15268sec preferred_lft 15268sec
    inet6 fd17:625c:f037:2:a4da:d21:4046:570a/64 scope global temporary dynamic
        valid_lft 86111sec preferred_lft 14111sec
    inet6 fd17:625c:f037:2:a00:27ff:feca:516c/64 scope global dynamic mngtmpaddr
        valid_lft 86111sec preferred_lft 14111sec
    inet6 fe80::a00:27ff:feca:516c/64 scope link
        valid_lft forever preferred_lft forever
kulani@kulani-VirtualBox:~/IE2102_IT24100368$
```

Figure 2.1: Network configuration

The screenshot shows the network interface configuration and the IP address assigned to the server machine. It also confirms that the system is networked appropriately to allow for the communication through the TCP socket for the assignment tasks.

3. Server running (port proof using ss/netstat)

```
kulani@kulani-VirtualBox:~/IE2102_IT24100368$ make -f Makefile_0368
make: Nothing to be done for 'all'.
kulani@kulani-VirtualBox:~/IE2102_IT24100368$ ./server_0368
[SID:1003] Server IT24100368 listening on port 50368

kulani@kulani-VirtualBox:~/IE2102_IT24100368$ ss -tlnp | grep 50368
LISTEN 0      20          0.0.0.0:50368  0.0.0.0:*    users:(("server_0368",pid=14635,fd=3))
kulani@kulani-VirtualBox:~/IE2102_IT24100368$
kulani@kulani-VirtualBox:~/IE2102_IT24100368$
```

Figure 3.1: Port proof

This screenshot confirms that the C server application is successfully compiled and listening for incoming TCP connections on the mandatory port 50368.

4. Successful REGISTER and LOGIN

```
kulani@kulani-VirtualBox:~/IE2102_IT24100368$ python3 client_0368.py
=====
IE2102 Network Programming Client - IT24100368
Server: 127.0.0.1:50368
=====
Commands: REGISTER <user> <pass>
          LOGIN <user> <pass>
          LOGOUT
          QUIT
=====
[+] Connected to 127.0.0.1:50368

client> REGISTER Kulani lore2003
server> OK 201 SID:1003 User Kulani registered successfully
```

Figure 4.1: Successfully register

```
kulani@kulani-VirtualBox:~/IE2102_IT24100368$ python3 client_0368.py
=====
IE2102 Network Programming Client - IT24100368
Server: 127.0.0.1:50368
=====
Commands: REGISTER <user> <pass>
          LOGIN <user> <pass>
          LOGOUT
          QUIT
=====
[+] Connected to 127.0.0.1:50368

client> LOGIN Kulani lore2003
server> OK 200 SID:1003 Login successful. TOKEN:1883b2fff453aaeacb7ee67dc81784524ad9dcb5330582498500c80247ee4a31
client>
```

Figure 4.2: Successfully login

The system implements a custom protocol where the client first sends a REGISTER command to create an account (Server response: 201 Created). Once registered, the user authenticates via a LOGIN command, prompting the server to validate credentials and issue a unique session token (Server response: 200 OK) to establish a secure connection.

5. Failed login + logout proof

```
kulani@kulani-VirtualBox:~/IE2102_IT24100368$ nano server_0368.c
kulani@kulani-VirtualBox:~/IE2102_IT24100368$ gcc -o server_0368 server_0368.c
kulani@kulani-VirtualBox:~/IE2102_IT24100368$ ./server_0368
[SID:1003] Server IT24100368 listening on port 50368

kulani@kulani-VirtualBox:~/IE2102_IT24100368$ python3 client_0368.py
=====
IE2102 Network Programming Client - IT24100368
Server: 127.0.0.1:50368
=====
Commands: REGISTER <user> <pass>
          LOGIN <user> <pass>
          LOGOUT
          QUIT
=====
[+] Connected to 127.0.0.1:50368

client> REGISTER Kulani lore2003
server> OK 201 SID:1003 User Kulani registered successfully
client> LOGIN Kulani lore2003
server> OK 200 SID:1003 Login successful. TOKEN:2d940ff70e4ac7f30a6822e9e97c74aec95b883ab407a4e46488116b83d3c474
client> LOGOUT
server> OK 200 SID:1003 Logged out successfully
client> LOGIN Kulani lor121
server> ERR 401 SID:1003 Invalid username or password
client> LOGIN Kulani lo233
server> ERR 401 SID:1003 Invalid username or password
client> LOGIN Kulani 234
server> ERR 401 SID:1003 Invalid username or password
client> LOGIN Kulani 675l
server> ERR 401 SID:1003 Invalid username or password
client> LOGIN Kulani lor756
server> ERR 423 SID:1003 Too many failed attempts. Locked for 5 min
client> LOGIN Kulani lor908
server> ERR 423 SID:1003 Account locked due to failed attempts
client>
```

Figure 5.1: Failed login + logout proof

The screenshot demonstrates the brute-force login protection implemented in `server_0368.c` for registration number `IT24100368`. The client first successfully registered a new user account. Then five consecutive failed login attempts were made using an incorrect password. After the 5th failed attempt, the server responded with `ERR 423` and locked the account for 5 minutes. A subsequent login attempt using the correct password was also rejected, confirming that the lockout mechanism is fully functional. This is implemented using a fail counter and a `locked_until` timestamp in the child process session, controlled by the `MAX_FAIL` (5 attempts) and `LOCKOUT_TIME` (300 seconds) constants defined in the server.

6. Multiple forked processes (ps output)

```
kulani@kulani-VirtualBox:~/IE2102_IT24100368$ make -f Makefile_0368
make: Nothing to be done for 'all'.
kulani@kulani-VirtualBox:~/IE2102_IT24100368$ ./server_0368
[SID:1003] Server IT24100368 listening on port 50368
```

```
kulani@kulani-VirtualBox:~/IE2102_IT24100368$ python3 client_0368.py
=====
IE2102 Network Programming Client – IT24100368
Server: 127.0.0.1:50368
=====
Commands: REGISTER <user> <pass>
          LOGIN <user> <pass>
          LOGOUT
          QUIT
=====
[+] Connected to 127.0.0.1:50368

client> REGISTER mary pass456
server> OK 201 SID:1003 User mary registered successfully
client>
```

```
kulani@kulani-VirtualBox:~/IE2102_IT24100368$ python3 client_0368.py
=====
IE2102 Network Programming Client – IT24100368
Server: 127.0.0.1:50368
=====
Commands: REGISTER <user> <pass>
          LOGIN <user> <pass>
          LOGOUT
          QUIT
=====
[+] Connected to 127.0.0.1:50368

client> REGISTER john pass123
server> OK 201 SID:1003 User john registered successfully
client> █
```

```
kulani@kulani-VirtualBox:~/IE2102_IT24100368$
kulani@kulani-VirtualBox:~/IE2102_IT24100368$ ps aux | grep server_0368
kulani    14635  0.0  0.0   2700  1680 pts/2    S+   00:06   0:00 ./server_0368
kulani    14799  0.0  0.0   2700  1572 pts/2    S+   00:13   0:00 ./server_0368
kulani    14880  0.0  0.0   2700  1572 pts/2    S+   00:15   0:00 ./server_0368
kulani    14902  0.0  0.0   9284  2332 pts/0    S+   00:16   0:00 grep --color=auto server_0368
kulani@kulani-VirtualBox:~/IE2102_IT24100368$
kulani@kulani-VirtualBox:~/IE2102_IT24100368$
kulani@kulani-VirtualBox:~/IE2102_IT24100368$
```

Figure 6.1: forked processes.

This task (A2 (Multiprocessing)) shows how the server can handle many client connections at once using Multiprocessing, which uses a fork() system call. When a client connects, a Child Process will be dedicated to handle only that particular client's request. The server will be able to handle other clients' requests without having to wait for the previous client's request to be served, thanks to this approach.

7. Log file entries

```
kulani@kulani-VirtualBox:~/IE2102_IT24100368$ cat server_IT24100368.log
[2026-04-11 22:21:47] CLIENT=127.0.0.1:34548 PID=11435 USER=- CMD=CONNECT RESULT=OK
[2026-04-11 22:22:36] CLIENT=127.0.0.1:34548 PID=11435 USER=- CMD=hello RESULT=AUTH_REQUIRED
[2026-04-11 22:26:56] CLIENT=127.0.0.1:34548 PID=11435 USER=kulani CMD=REGISTER RESULT=SERVER_ERROR
[2026-04-11 22:40:33] CLIENT=127.0.0.1:55222 PID=12466 USER=- CMD=CONNECT RESULT=OK
[2026-04-11 22:40:49] CLIENT=127.0.0.1:55222 PID=12466 USER=kulani CMD=REGISTER RESULT=OK
[2026-04-11 22:41:14] CLIENT=127.0.0.1:55222 PID=12466 USER=kulani CMD=LOGIN RESULT=OK
[2026-04-11 22:41:23] CLIENT=127.0.0.1:55222 PID=12466 USER=kulani CMD=hello RESULT=UNKNOWN_CMD
[2026-04-11 22:41:51] CLIENT=127.0.0.1:55222 PID=12466 USER=kulani CMD=LOGOUT RESULT=OK
[2026-04-11 22:41:56] CLIENT=127.0.0.1:55222 PID=12466 USER=- CMD=QUIT RESULT=DISCONNECTED
[2026-04-11 22:41:56] CLIENT=127.0.0.1:55222 PID=12466 USER=- CMD=DISCONNECT RESULT=OK
[2026-04-11 23:31:03] CLIENT=127.0.0.1:51824 PID=12942 USER=- CMD=CONNECT RESULT=OK
[2026-04-11 23:31:23] CLIENT=127.0.0.1:51824 PID=12942 USER=test CMD=LOGIN RESULT=FAILED
[2026-04-11 23:31:39] CLIENT=127.0.0.1:51824 PID=12942 USER=test CMD=LOGIN RESULT=FAILED
[2026-04-11 23:31:42] CLIENT=127.0.0.1:51824 PID=12942 USER=test CMD=LOGIN RESULT=FAILED
[2026-04-11 23:31:59] CLIENT=127.0.0.1:51824 PID=12942 USER=kulani CMD=LOGIN RESULT=FAILED
[2026-04-11 23:32:14] CLIENT=127.0.0.1:51824 PID=12942 USER=kulani CMD=LOGIN RESULT=LOCKOUT
[2026-04-11 23:32:30] CLIENT=127.0.0.1:42254 PID=12992 USER=- CMD=CONNECT RESULT=OK
[2026-04-11 23:32:42] CLIENT=127.0.0.1:58210 PID=13001 USER=- CMD=CONNECT RESULT=OK
[2026-04-11 23:35:34] CLIENT=127.0.0.1:58210 PID=13001 USER=- CMD=DISCONNECT RESULT=OK
[2026-04-11 23:47:31] CLIENT=127.0.0.1:36510 PID=14029 USER=- CMD=CONNECT RESULT=OK
[2026-04-11 23:48:24] CLIENT=127.0.0.1:36510 PID=14029 USER=kulani CMD=REGISTER RESULT=OK
[2026-04-11 23:49:34] CLIENT=127.0.0.1:36510 PID=14029 USER=kulani CMD=LOGIN RESULT=OK
[2026-04-11 23:49:57] CLIENT=127.0.0.1:36510 PID=14029 USER=kulani CMD=LOGOUT RESULT=OK
[2026-04-11 23:53:16] CLIENT=127.0.0.1:36510 PID=14029 USER=kulani CMD=LOGIN RESULT=FAILED
[2026-04-11 23:53:35] CLIENT=127.0.0.1:36510 PID=14029 USER=kulani CMD=LOGIN RESULT=FAILED
[2026-04-11 23:53:43] CLIENT=127.0.0.1:36510 PID=14029 USER=kulani CMD=LOGIN RESULT=FAILED
[2026-04-11 23:53:50] CLIENT=127.0.0.1:36510 PID=14029 USER=kulani CMD=LOGIN RESULT=FAILED
[2026-04-11 23:54:03] CLIENT=127.0.0.1:36510 PID=14029 USER=kulani CMD=LOGIN RESULT=LOCKOUT
[2026-04-11 23:54:30] CLIENT=127.0.0.1:36510 PID=14029 USER=- CMD=LOGIN RESULT=LOCKED
kulani@kulani-VirtualBox:~/IE2102_IT24100368$
```

Figure 7.1: Comprehensive Server Audit Log (server_IT24102161.log).

The server uses a persistent logging mechanism, where all critical events, including successful, failed, and locked-out attempts, are logged. Each entry in the log contains information such as Timestamp, Client's IP Address, Action taken, and Status Code. The server's log file, server_IT24100368.log, contains all client activities, including timestamp, IP:port, PID, username, command, and result, thus ensuring all A5 requirements have been met.

8. Proper child cleanup (no zombie processes)

```
client>
client> LOGOUT
server> OK 200 SID:1003 Logged out successfully
client> █
```

```
kulani@kulani-VirtualBox:~/IE2102_IT24100368$
kulani@kulani-VirtualBox:~/IE2102_IT24100368$ ps aux | grep defunct
kulani    14929  0.0  0.0   9152  2324 pts/0    S+   00:16   0:00 grep --color=auto defunct
kulani@kulani-VirtualBox:~/IE2102_IT24100368$ █
```

Figure 8.1: Zombie Process check

Process lifecycle management is very important in a multiprocessor server. As shown in the following screen shot, upon termination of the client connections, memory leaks in the server are prevented, i.e., the server doesn't create zombie processes. The following is achieved through implementation of the SIGCHLD handler in which waitpid() system call.